

✓ 00 — Project overview

I define the scientific question, repository contract, and execution plan for my V1 natural movie manifold project. I keep this notebook lightweight because the later notebooks perform the real analysis.

```
from pathlib import Path
import sys

ROOT = Path.cwd()
if ROOT.name == "notebooks":
    ROOT = ROOT.parent
if str(ROOT / "src") not in sys.path:
    sys.path.insert(0, str(ROOT / "src"))

from v1_manifold.config import load_config, get_paths, set_global_seed
from v1_manifold.visualization import set_publication_style, save_figure
from v1_manifold.utils import save_table

cfg = load_config(ROOT / "configs" / "default.yaml")
cfg["paths"]["root"] = str(ROOT)
paths = get_paths(cfg)
set_global_seed(cfg["project"]["random_seed"])
set_publication_style()
print(f"Project root: {ROOT}")
```

Project root: c:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-na

```
import json
from pprint import pprint

summary = {
    "project": cfg["project"]["name"],
    "stimulus": cfg["preprocessing"]["stimulus_name"],
    "targeted_structures": cfg["allen"]["targeted_structures"],
    "cre_lines": cfg["allen"]["cre_lines"],
    "default_max_experiments": cfg["allen"]["max_experiments"],
}
pprint(summary)
(paths.tables_dir / "00_project_configuration.json").write_text(json.dumps(summary))
```

```
{'cre_lines': ['Cux2-CreERT2',
               'Cux2-CreERT2;Camk2a-tTA;Ai93',
               'Rorb-IRES2-Cre',
               'Rbp4-Cre_KL100',
               'Ntsr1-Cre_GN220',
```

```

        'Emx1-IRES-Cre'],
    'default_max_experiments': 3,
    'project': 'latent-manifold-v1-natural-movies',
    'stimulus': 'natural_movie_one',
    'targeted_structures': ['VISp']}]
327

```

✓ Research design

I treat each Allen ophys experiment as a session-level neural population. I first build repeat-aligned natural movie response tensors, then compare raw/PCA/nonlinear latent geometries, then evaluate stimulus decodability and model-to-brain alignment. The default configuration is small enough for a laptop, while the same code scales to more sessions.

```

import pandas as pd

workflow = pd.DataFrame([
    {"stage": "01", "notebook": "data acquisition", "main_output": "Allen ex"},
    {"stage": "02", "notebook": "loading and validation", "main_output": "sh"},
    {"stage": "03", "notebook": "preprocessing", "main_output": "trial tenso"},
    {"stage": "04", "notebook": "EDA", "main_output": "population activity f"},
    {"stage": "05", "notebook": "latent representations", "main_output": "PC"},
    {"stage": "06", "notebook": "baseline decoders", "main_output": "classic"},
    {"stage": "07", "notebook": "CEBRA and dRNN", "main_output": "advanced e"},
    {"stage": "08", "notebook": "model comparison", "main_output": "benchmar"},
    {"stage": "09", "notebook": "geometry and alignment", "main_output": "cu"},
    {"stage": "10", "notebook": "report generation", "main_output": "structu"}
])
save_table(workflow, paths.tables_dir / "00_workflow_plan.csv")
workflow

```

	stage	notebook	main_output
0	01	data acquisition	Allen experiment catalog
1	02	loading and validation	shape/schema checks
2	03	preprocessing	trial tensors
3	04	EDA	population activity figures
4	05	latent representations	PCA/UMAP/ISOMAP embeddings
5	06	baseline decoders	classical decoding benchmark
6	07	CEBRA and dRNN	advanced embeddings/checkpoints
7	08	model comparison	benchmark tables

8	09	geometry and alignment	curvature/tangling/CKA/RSA
9	10	report generation	structured summary